FINSIGHT AI

OpenBao for Local Secret Management

Project-specific architecture, integration plan, security controls, and FIN-14 acceptance evidence

Executive recommendation

Adopt OpenBao as the self-hosted, local secret-management service for FinSight AI. It is a strong fit for the no-cloud requirement, but it is a proposed integration—not yet an implemented or verified provider in the supplied evidence. Preserve the existing SecretStore interface and replace the development-only provider with an OpenBao adapter.

Field	Value
Document status	Architecture proposal and implementation gap analysis
Prepared	27 October 2026
Application context	Local Mac implementation; FastAPI + PostgreSQL; no cloud deployment planned
Security milestone	FIN-14 / MFA production secret-store readiness
Evidence boundary	Project source documents and implementation/test checkpoints supplied in this conversation; latest execution checkpoint found is dated 23 October 2026

Prepared to support engineering implementation, design review, and acceptance testing.

Contents

- 1. Executive summary and decision
- 2. FinSight AI context and requirements mapping
- 3. OpenBao: purpose, boundaries, and fit
- 4. Current implementation status versus target state
- 5. Target architecture specific to FinSight AI
- 6. MFA data model and secret-path design
- 7. End-to-end MFA lifecycle and failure behavior
- 8. Local OpenBao deployment and configuration design
- 9. FastAPI integration plan mapped to the repository
- 10. Security, operations, backup, and recovery controls
- 11. FIN-14 acceptance and test evidence matrix
- 12. Implementation sequence and definition of done
- 13. Risks, trade-offs, and residual limitations
- 14. Conclusion
- Appendix A. Illustrative policy and configuration variables
- Appendix B. References and source traceability

How to read implementation status

“Implemented” means the supplied project evidence shows code or tests for it. “Proposed” means this report recommends it but the provided materials do not prove it is configured or running. Test pass counts are local evidence only and do not prove OpenBao integration or production readiness.

Document control and terminology

Term	Meaning in this report
OpenBao	Self-hosted open-source secrets-management service selected as the target provider.
MFA / TOTP seed	A user-factor-specific secret used by an authenticator app to generate time-based one-time codes.
SecretStore interface	The provider-neutral Python contract already present in the supplied project evidence: put, get, delete.
FIN-14	The project implementation/security checkpoint discussed in the supplied status notes. Do not confuse it with SRS identifier FR-P1-14, which covers the read-versus-trading permission boundary.
KV v2	OpenBao versioned key-value secrets engine; recommended for TOTP seeds and static API credentials.

1. Executive summary and decision

OpenBao is the recommended secret-management component for FinSight AI because the project is intended to remain a local implementation, uses Python FastAPI for the application API, and needs a real secret-store provider to close the gap currently recorded for MFA production readiness. OpenBao provides a self-hosted API, path-based access policies, versioned KV storage, an encrypted storage barrier, audit devices, and machine-oriented authentication methods. It does not require the FinSight application to deploy to a cloud platform. [5]-[11]

Decision

Use one persistent local OpenBao instance with integrated Raft storage, KV v2 mounted at `secret/`, AppRole (or a comparably constrained machine identity), a narrow MFA policy, TLS on loopback, and an enabled audit device. Keep the root token and unseal/recovery material strictly in the operator workflow, never in FastAPI.

Question	Finding
Is OpenBao a fit?	Yes. It aligns with self-hosting, open-source control, local operation, and the existing provider-neutral SecretStore interface.
What role does it play?	Stores and retrieves sensitive values, especially TOTP seeds, and optionally provider API credentials; enforces a machine policy and records secret-store API access.
Is it implemented already?	Not according to the supplied evidence. MFA routes and provider-neutral storage are present; the production selector rejects production because no production provider is configured. No OpenBao server, policy, AppRole, adapter, or integration-test evidence was supplied.
What is the current FIN-14 status?	PARTIAL / not accepted. The local MFA implementation has test coverage, but OpenBao and live secret-store checks remain to be implemented and evidenced.
Does OpenBao make the project automatically secure?	No. The application must enforce ownership, MFA policy, replay protection, safe errors, and cleanup. The operator must manage initialization, unseal keys, TLS, audit log protection, backups, and restore drills.

Expected outcome

- TOTP seed material is held in OpenBao, not PostgreSQL rows, source code, `.env` files, or logs.
- PostgreSQL keeps MFA factor state and a non-secret reference to the OpenBao path.
- FastAPI uses a narrowly scoped machine identity and never uses the OpenBao root token.
- Provider failure returns a controlled error and never silently falls back to plaintext or disables MFA.
- FIN-14 is only marked accepted after the OpenBao-backed flow, access policy, audit, failure, rotation, backup, and recovery checks have evidence.

2. FinSight AI context and requirements mapping

The supplied SRS defines FinSight AI as a locally oriented AI workspace with a frontend, a backend for authentication and business logic, PostgreSQL for structured state, a local or configured Ollama runtime, and RAG and analysis capabilities. Its authentication area includes MFA, authentication-event recording, brute-force controls, active session management, protected routes, and backend permission enforcement. The technical design adds the

explicit security rule that a secret manager/environment layer holds credentials while the database stores references or hashes rather than plaintext secrets. [1, pp. 1, 3-5; 2, pp. 6-9, 14]

Source requirement / design rule	OpenBao contribution	What remains in FinSight code
SRS FR-P1-11: MFA, when enabled by policy	Protected storage of TOTP seed; access policy for the backend identity.	Enrollment/activation policy, challenge lifecycle, TOTP validation, replay protection, session issue.
SRS FR-P1-09: authentication event recording	Audit records for OpenBao API requests and outcomes.	Application audit for login, MFA activation/revocation, actor, factor, outcome, and correlation ID.
SRS FR-P1-10: rate limit / lockout	Not a replacement for rate limiting.	Existing FastAPI/PostgreSQL throttle and lockout logic remains authoritative.
SRS FR-P1-12: active session management	Not a session manager.	Session listing/termination, expiry, token invalidation and tests remain application responsibilities.
Technical design: DB stores references/hashes, not secret values	Holds actual TOTP seed and other static credentials behind the API.	Persist `secret_ref` and factor metadata; verify no seed or provisioning URI is persisted or logged.
NFR-P1-02: HTTPS in deployed environments	TLS-secured API connection; local listener should also use TLS in the production-like local profile.	Trust the correct local CA/certificate and reject invalid certificates. Do not disable TLS verification.

Identifier clarification

FIN-14 is used in the supplied project checkpoint notes for the MFA/secret-store acceptance work. It is distinct from SRS FR-P1-14, which specifies separation between read-only data access and trading permissions.

3. OpenBao: purpose, boundaries, and fit

3.1 What it is

OpenBao is a secrets-management server. An application authenticates to it, obtains a token associated with defined policies, and calls the API to read or write specific secret paths. KV v2 stores versioned key-value data. OpenBao encrypts data through its storage barrier before persisting it to the backend. Audit devices can record requests and responses for interactions with the API. [5]-[9]

3.2 What FinSight will use it for

- TOTP seed for each MFA factor: the highest-priority FIN-14 use case.
- Potentially, financial market-data provider API keys and other static integration credentials as provider modules are completed.
- Potentially, other static runtime credentials that must not be committed to Git or exposed to the frontend.
- Versioned secret lifecycle and an access trail that complements—not replaces—the application audit trail.

3.3 What it does not do

- It does not authenticate FinSight users or verify TOTP codes by itself.
- It does not replace PostgreSQL, user/factor metadata, database migrations, sessions, login throttling, RBAC, or correlation IDs.
- It does not automatically rotate TOTP authenticators safely; MFA re-enrollment is an application workflow.

- It does not automatically guarantee immutable audit evidence, availability, backup correctness, or successful recovery.
- It does not protect a seed from a fully compromised FastAPI process that has permission to read the entire MFA prefix.

3.4 Why OpenBao instead of cloud Secret Manager or only an encrypted file?

Option	Advantages	Limitations for this project	Decision
OpenBao local service	Open source; owned configuration; policy-based API; KV versions; audit devices; no vendor cloud requirement.	Adds an operations component: unseal, TLS, ACL, tokens, data protection, backup and recovery.	Recommended target
Existing encrypted development file backend	Minimal moving parts; already reported as development-only; testable locally.	Secret handling depends on local filesystem/key management; not a central policy/audit service; not a production provider in current project status.	Keep for development only or remove after migration
Cloud Secret Manager	Managed operations and IAM integration.	Contradicts the stated no-cloud deployment direction and introduces a separate external dependency/pricing model.	Not selected
Fake in-memory test backend	Fast, deterministic and offline tests.	Data is not durable and must never be used for real user MFA.	Keep for automated tests

Cost and ownership

OpenBao avoids a paid cloud secret-manager dependency and does not require a subscription for the open-source software. “Free” does not mean zero operational cost: the project owns the security, patching, backup, recovery, and access-control workload.

4. Current implementation status versus target state

The status below is based on the project code/log excerpts and traceability notes supplied in the conversation. The latest explicit MFA test checkpoint found in that evidence is dated 23 October 2026; a current repository checkout was not provided as an executable workspace in this task. Treat these items as supplied evidence and re-check the live branch before changing implementation state.

Component	Status supported by supplied evidence	OpenBao relevance / gap
FastAPI MFA endpoints	Routes are present for MFA enrollment, activation, login verification, factor listing and factor revocation.	Retain routes; route through the existing service and provider-neutral interface.
MFA metadata schema	`mfa_factors` includes factor UUID, user UUID, type, label, `secret_ref`, factor status (`pending`, `active`, `revoked`) and timestamps.	Suitable foundation; consider a `secret_version` column if the design pins versions explicitly.
SecretStore contract	`put(secret)`, `get(ref)`, and `delete(ref)` are present in `apps/api/app/secret_store.py`.	Extend/adapt the contract only if OpenBao needs explicit version and health/auth methods. Avoid provider-specific code inside routes.
Test provider	A fake in-memory store exists for automated tests.	Keep it; unit tests must not require an OpenBao service.

Component	Status supported by supplied evidence	OpenBao relevance / gap
Development provider	An encrypted file backend using AES-GCM is described as development-only.	Do not silently promote it to production or use it as fallback after OpenBao failure.
Production provider selector	Supplied release notes say `APP_ENV=production` is deliberately rejected until a production provider is configured.	Implement and test OpenBao adapter, configuration validation and production selection.
Test evidence	Checkpoint reports focused MFA/repository tests: 30 passed; full API suite: 150 passed, 0 failed, 25 warnings.	Good evidence for local MFA components, but not evidence of an OpenBao server, policy, audit device or backup restore.
OpenBao service / policy / TLS / AppRole	No configuration or passing integration evidence for these items was found in the supplied project snippets.	Not implemented/verified in this report; acceptance evidence is still required.

Honest conclusion on present state

MFA implementation exists in the application; the production secret provider is not yet configured according to the supplied status evidence. Therefore FIN-14 remains PARTIAL, and OpenBao must be treated as the next implementation milestone rather than a completed feature.

5. Target architecture specific to FinSight AI

The target is a local modular application, not a microservice decomposition exercise. OpenBao is a separate local security process because it has its own data-protection barrier, token policy engine, audit device and lifecycle. The Next.js frontend must never receive an OpenBao token or call OpenBao directly. Only the FastAPI process should access the OpenBao API using its restricted machine identity.

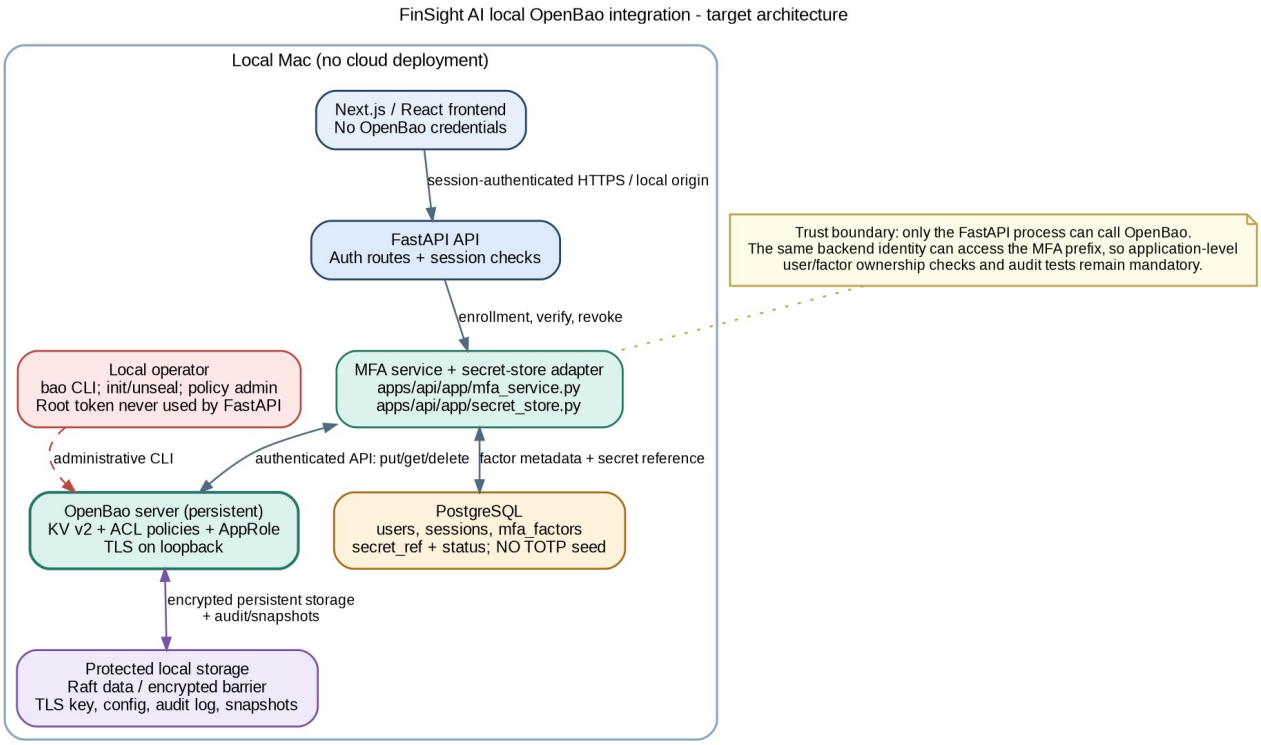


Figure 1. Proposed deployment boundary and trust relationships for the local Mac environment.

5.1 Component responsibilities

Component	Owns	Must not own / must not do
Next.js / React	Enrollment UI, QR display, code entry, session-based calls to FastAPI.	Never store OpenBao tokens, direct secret paths, TOTP seeds after enrollment, or provider API keys in browser storage.
FastAPI route layer (`main.py`)	Request validation, session/origin checks, correlation IDs, authorization and safe HTTP error mapping.	Never expose secret values in responses/logs except the one-time provisioning URI/seed needed for authenticator enrollment.
MFA service (`mfa_service.py`)	Orchestrates seed generation, OpenBao calls, TOTP verification, factor state transitions, cleanup and replay checks.	Must not implement OpenBao HTTP path semantics inline in every route.
Secret-store adapter (`secret_store.py` or `openbao_secret_store.py`)	Provider API calls, TLS verification, AppRole login/token handling, JSON parsing and provider error translation.	Must not authorize cross-user factor operations alone; must not log request payload secrets.
PostgreSQL / repositories	User/factor/session/challenge metadata, references, status, replay guard and application audit events.	Must not contain raw TOTP seed or enrollment URI. `secret_ref` must not be treated as an authorization credential.
OpenBao	Durable secret persistence, encryption barrier, policy enforcement, secret versioning, OpenBao API audit.	Does not know FinSight user permissions unless the application enforces them or a future identity-bound architecture is added.
Local operator / CLI	Initialization, unseal, policy administration, upgrades, backup, recovery and audit access.	Must not use root token in the API process or commit unseal material, role secret IDs or tokens.

5.2 Trust boundaries and important limitation

The FastAPI machine identity will need access to multiple users' MFA records because it verifies login challenges for those users. Consequently, a compromised FastAPI process could read every seed within its allowed OpenBao path. Path-based least privilege still limits that identity to FinSight MFA secrets rather than all OpenBao data, but it is not per-user isolation. Keep the token unavailable to the browser; validate factor ownership in the repository/service; keep the policy narrow; monitor accesses; and consider a separate authentication process only if stronger isolation becomes a project requirement.

6. MFA data model and secret-path design

The supplied implementation snippet shows an `mfa_factors` table with `secret_ref` and states `pending`, `active`, and `revoked`. This is a good separation between relational metadata and secret payload. Use immutable, factor-specific paths so one user's second authenticator is not confused with their first and a seed change does not overwrite a live factor.

Field / path	Recommended meaning	Notes
`mfa_factors.mfa_factor_id`	Unique factor UUID.	Use it in the secret path; never use email as a path component.
`mfa_factors.user_id`	Owner UUID.	The owner must be validated before get/revoke operations.

Field / path	Recommended meaning	Notes
`mfa_factors.secret_ref`	Logical OpenBao reference, e.g. `finsight/mfa/<user_uuid>/<factor_uuid>`.	Reference is not a credential. Build the KV v2 API path inside the adapter.
Optional `secret_version`	Positive integer version if exact-version pinning is stored separately.	Add with a versioned migration if chosen. For TOTP, prefer immutable path-per-factor and do not update the seed in-place.
`factor_status`	`pending` until code confirmation; `active` after confirmation; `revoked` immediately on successful owner-authorized revocation.	Do not enable a pending factor before checking a valid TOTP code.
OpenBao KV v2 API path	`secret/data/finsight/mfa/<user_uuid>/<factor_uuid>`.	KV v2 includes the `data/` segment in its API path. The application stores only the logical reference.

6.1 Suggested layout

```
OpenBao mount: secret/              (KV version 2)
Logical path:  finsight/mfa/<user_uuid>/<factor_uuid>
API path:      secret/data/finsight/mfa/<user_uuid>/<factor_uuid>
KV value:      { "totp_seed": "<secret bytes>" }

PostgreSQL mfa_factors row:
  mfa_factor_id = <factor UUID>
  user_id       = <owner UUID>
  factor_type   = 'totp'
  secret_ref    = 'finsight/mfa/<user_uuid>/<factor_uuid>'
  factor_status = 'pending' | 'active' | 'revoked'

Never store the TOTP seed or full otpauth:// provisioning URI in PostgreSQL.
```

Do not name secrets by email address

Use generated internal UUIDs, not user-controlled strings, email addresses, or display names. This avoids personal information in secret paths and reduces path-injection/encoding mistakes.

7. End-to-end MFA lifecycle and failure behavior

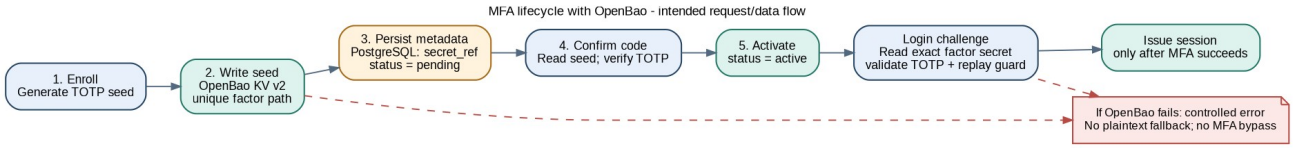


Figure 2. Enrollment, activation and login flow; the same secret-store adapter is used for all seed access.

7.1 Enrollment

1. Authenticate the user and apply step-up checks if required by policy. Generate a cryptographically random TOTP seed in the MFA service.
2. Write the seed to a unique OpenBao KV v2 path using the restricted application identity.

3. Persist `secret_ref` and a `pending` factor row in PostgreSQL. Because OpenBao and PostgreSQL do not share a transaction, define compensating cleanup: if the DB write fails, attempt to delete the new OpenBao data; if cleanup fails, retain a retryable cleanup record/reference.
4. Return the provisioning URI or QR payload only to the authenticated user for enrollment. Do not write it into logs, analytics, audit payloads, or error traces.

7.2 Activation

1. Read the pending factor's seed from OpenBao through the adapter and verify the submitted TOTP code with the existing TOTP library.
2. Only after a valid code is confirmed, transition the factor from `pending` to `active` with an ownership-checked database update and an application audit event.
3. Reject activation if the factor belongs to another user, is already revoked, or its secret cannot be retrieved. Do not activate a factor based only on successful secret creation.

7.3 Login with MFA

1. Validate the password according to the existing authentication code. If the user has an active MFA factor under the configured policy, create a short-lived MFA challenge instead of issuing a fully authenticated session.
2. On code submission, validate challenge ownership and expiry, retrieve the factor seed, and verify the TOTP code. Apply the existing replay guard and throttling rules.
3. Issue the final application session only after MFA succeeds. Store only session/token hashes or opaque references according to the current session design; do not log secret-store tokens or the TOTP seed.

7.4 Factor revocation and cleanup

1. Require an authenticated owner (and step-up authentication where appropriate); reject cross-user revocation before calling OpenBao.
2. Mark the factor revoked so it cannot be used for future challenges. Attempt to remove/soft-delete the secret through the provider adapter.
3. If secret deletion fails, keep the secret reference or cleanup job state so a retry can succeed. On successful cleanup, clear the reference as the supplied implementation checkpoint already describes.
4. Record factor ID, actor, result, correlation ID, and provider result category in application audit logs, but never include the seed or provider token.

7.5 Rotation and recovery

A TOTP seed is tied to a user's authenticator. Rotation is therefore a new-factor enrollment: create a new factor path, let the user confirm it, atomically mark the new factor active, and then revoke the old factor. Do not simply overwrite `latest` under the old path: that can invalidate the authenticator without the user confirming the replacement. API keys can use ordinary version rotation, but each credential's application cutover and rollback behavior must be designed separately.

7.6 Failure matrix

Failure	Required application behavior	Evidence to collect
OpenBao sealed / down / timeout	Fail closed for MFA-protected login. Return safe `503`-class dependency error; never skip MFA or fall back to file/DB seed.	Failure-injection test; response code; logs show correlation ID and no secret.
AppRole rejected / token expired	Renew or re-authenticate through the supported flow; bounded retry only; deny operation while unauthorized.	Invalid/expired token and recovery tests.

Failure	Required application behavior	Evidence to collect
Secret missing / disabled / invalid version	Deny the protected action and route user through controlled recovery/re-enrollment.	Not-found/version tests; no error message reveals secret data.
PostgreSQL unavailable after OpenBao write	Compensate by cleanup or create a retryable orphan-reconciliation record.	Integration test with DB failure injection.
OpenBao delete unavailable during factor revocation	Keep factor revoked; preserve cleanup reference/job; retry cleanup; do not reactivate factor.	Retry behavior test and final reference-clearing assertion.
Audit device unavailable or disk full	Treat as an operationally critical event; define whether secret operations fail closed according to security policy, alert the operator, and do not falsely claim audit evidence exists.	Audit-device failure drill and documented policy.

8. Local OpenBao deployment and configuration design

Use a persistent single-node OpenBao service on the Mac. Integrated Raft storage provides a durable local backend and snapshot/restore operations; a one-node setup does not provide high availability. OpenBao's storage barrier encrypts data before persistence, but protection still depends on safeguarding initialization/unseal material, the local account, configuration, TLS private keys, and backups. [6], [10], [11]

Control	Recommended local configuration	Why it matters
Server process	Run OpenBao as a dedicated local service/process with documented start/stop commands.	Reproducible startup and auditable ownership.
Listener	Bind only to `127.0.0.1:8200` for this local-only project; use TLS with certificate SAN matching `localhost`/loopback.	Prevents LAN exposure and encrypts API traffic. Never use `tls_skip_verify` / `verify=False`.
Storage	Integrated Raft storage in a data directory outside the Git repository with owner-only filesystem permissions.	Durable state across restart; no development-mode in-memory data.
Initialization / unseal	Initialize once; store Shamir unseal shares/recovery materials outside the app and separate from the main data disk. Follow the documented unseal routine after restart.	Without unseal material, encrypted storage cannot be opened; losing it can make secrets unrecoverable.
Root token	Use only for initial setup and emergency administration; revoke/secure according to the documented operating model after creating policy and AppRole.	Root privileges would bypass the separation that OpenBao is intended to provide.
Machine authentication	Use AppRole for the FastAPI machine identity; short-lived, narrowly scoped tokens; protect secret ID in macOS Keychain or a permission-restricted secret file outside Git.	Avoid hardcoded long-lived root tokens and keep service credentials separate from user sessions.
Audit device	Enable a file audit device under a protected log directory; ensure file permissions and log rotation; test secret-value redaction in actual logs.	Provides a record of OpenBao API access and errors. Local file permissions do not make logs immutable against a Mac administrator.
Backups	Take authenticated Raft snapshots and back up configuration/certificates. Keep a protected copy on encrypted external storage; store unseal recovery material separately.	Protect against disk loss and accidental corruption without using cloud storage.

Control	Recommended local configuration	Why it matters
Network / OS	Use a dedicated unprivileged OS account where practical; file permissions `0700` for data directories and restrictive read/write for TLS keys, audit logs and credentials.	Reduce accidental exposure to other local processes and users.

Do not use development mode

The OpenBao `-dev` server is for experimentation only: it uses insecure, ephemeral defaults and should not store real MFA seeds. A persistent, initialized, unsealed server with TLS and policies is required for the target architecture. [10]

8.1 Operator setup sequence

1. Install a current supported OpenBao release from the official installation instructions and verify the downloaded release/checksum. Pin the tested version in the project documentation.
2. Create the configuration, TLS certificate/CA trust, and persistent storage directory outside the source tree. Bind the listener to loopback only.
3. Start the service and initialize it once. Securely capture unseal shares/recovery material and initial admin token; never paste these values into project documentation, terminal recordings, or chat.
4. Unseal the server. Enable `secret/` as KV v2, create the restricted MFA policy, enable AppRole, configure the FastAPI role, and enable the audit device.
5. Create a non-sensitive canary secret and test approved read/write/delete operations and denied operations using the application identity. Delete the canary after verification.
6. Configure FastAPI to use OpenBao only when the provider is explicitly selected and all required settings validate. In production-like mode, missing configuration must stop startup or cause a deliberate unhealthy/closed state, not select the development file provider.
7. Reboot/restart the service and prove that data persists, the operator can unseal, the API can authenticate again, and a backup can be restored to an isolated test data directory.

9. FastAPI integration plan mapped to the repository

The least disruptive path is to retain the provider-neutral `SecretStore` abstraction. MFA routes and domain rules should call `mfa_service.py`; the provider adapter alone knows OpenBao HTTP API paths, KV v2 response shape, authentication, timeout behavior and error mapping. This preserves the fake provider for tests and allows a deliberate migration from the development file backend.

Repository area from supplied evidence	Change required	Acceptance result
`apps/api/app/secret_store.py`	Keep the Protocol (`put`, `get`, `delete`); define clear exceptions and, if needed, explicit secret-version/path metadata.	Providers are interchangeable; adapter errors are typed and secret payloads never enter logs.
New `apps/api/app/openbao_secret_store.py` (recommended)	Implement TLS-verified HTTP calls to OpenBao KV v2; authenticate by AppRole; cache/renew only short-lived token safely; validate paths; map 403/404/5xx/timeouts.	OpenBao-backed contract tests pass against the local server.
`apps/api/app/mfa_service.py`	Keep TOTP generation/validation and factor lifecycle here; call the abstract store. Add robust compensation/reconciliation behavior for cross-system writes/deletes.	Enrollment, activation, login challenge, revoke and cleanup behavior pass end-to-end tests.

Repository area from supplied evidence	Change required	Acceptance result
`apps/api/app/mfa_repository.py` and challenge repository	Keep `secret_ref`, state changes, ownership checks, challenge expiry and replay guard in PostgreSQL; consider a migration for pinned `secret_version` if required.	DB contains only metadata/reference; cross-user operations are denied.
`apps/api/app/main.py`	Keep existing routes. Ensure dependency failure maps to a controlled status, correlation ID is preserved, and no provider details/secrets leak to clients.	Existing auth/security tests remain green plus new OpenBao API integration tests.
`apps/api/requirements.txt` and lock file	Add the chosen HTTP client only if absent; lock dependencies; avoid assuming a Vault client library is fully compatible without testing.	Reproducible install and lock consistency.
`.env.example` / local config docs	Add placeholders for OpenBao address, CA certificate, mount/prefix, AppRole role ID, secure secret-ID source, and timeouts. Do not add real tokens or IDs.	Clean clone instructions work without committed credentials.
Migrations and tests	Use additive SQL migration only if required for pinned version, cleanup outbox/reconciliation, or audit fields. Add OpenBao integration tests with a dedicated local test instance.	Schema upgrade, failure rollback, and acceptance evidence are reproducible.

9.1 Adapter behavior and contract

Operation	OpenBao behavior	Adapter requirements
`put(secret)`	Write seed to `secret/data/finsight/mfa/<user>/<factor>` using KV v2 JSON body.	Validate identifiers; TLS verify; no seed logging; return logical reference and, if the contract supports it, version metadata.
`get(ref)`	Read only the exact expected path and parse KV v2 `data.data` shape.	Validate prefix against an allow-list; never accept arbitrary URLs/paths from client input; map denied/missing/sealed errors safely.
`delete(ref)`	Soft-delete/disable data for that factor according to the chosen KV v2 lifecycle.	Idempotent behavior; distinguish already missing from genuine provider outage; don't grant general metadata destroy to runtime role.
Auth/token handling	AppRole login returns a client token constrained by the policy.	Protect secret ID; maintain token only in process memory; renew/re-auth before expiry; no token in logs, exception traces or API responses.
Health	Check service health and token state using limited API endpoints.	Expose only readiness/dependency status; don't make general secret paths available through health APIs.

10. Security, operations, backup, and recovery controls

10.1 Least-privilege model

Identity	Allowed	Explicitly not allowed
FinSight API AppRole token	Create/read/update/soft-delete only under `secret/data/finsight/mfa/*` as required by the current lifecycle.	Root/system endpoints, enabling engines, modifying policies, reading unrelated provider namespaces, listing all secrets, destroying all historical versions.
Operator/admin	Initialize/unseal, create policy/role, manage audit devices, upgrades and recovery.	Routine application runtime access; committing root/unseal/secret ID materials.
End user/browser	Call FinSight MFA endpoints under the logged-in application session.	Any OpenBao address/token/path credential or direct OpenBao request.
Automated tests	Use `FakeSecretStore` for unit tests; isolated OpenBao instance for dedicated integration tests.	Production secrets, real user accounts, production tokens.

10.2 Audit logging and immutability

OpenBao audit devices can record API interactions and errors, and multiple audit devices can provide redundancy. [9] Enable at least one file device for the local setup and test which fields are HMACed/redacted by the installed version. Keep the application audit table as well: it answers who attempted to enroll, activate, authenticate with, or revoke a FinSight factor, while OpenBao audit records the service's secret-store operations.

> **Important: audit is not automatically immutable**
>
> A local audit file writable by the same OS administrator can be altered or deleted. To claim tamper resistance, separate service and operator permissions, restrict file writes, rotate and checksum/verify logs, maintain protected offline copies, and document the privileged-administrator threat. If formal immutability is required, a local-only deployment must define an independently protected append-only/immutable evidence destination; OpenBao alone does not solve that boundary.

10.3 Backup and recovery

- Back up OpenBao's persistent Raft data using the documented snapshot procedure; do not copy a live Raft directory casually as if it were a consistent backup. Offline backups are preferred where practical. [11]
- Back up PostgreSQL as well. OpenBao's data and `mfa_factors.secret_ref` records must be restored consistently; otherwise factor rows may point to missing secrets, or orphan secrets may remain.
- Include OpenBao configuration, TLS CA/certificate material, audit-device configuration, version/policy definitions and a secure record of the procedure needed to unseal/recover.
- Store recovery material separately from the primary Mac and data backup. A backup without the ability to unseal/restore is not a tested recovery plan.
- Perform a restore drill into an isolated directory/database, then retrieve a test secret using a test identity. Verify that denied access remains denied after restoration.
- The technical design asks the deployment owner to define RTO/RPO; the supplied source does not assign numeric targets. Select and document targets for this local project rather than inventing them in the implementation report. [2, p. 15, p. 24]

10.4 Lifecycle maintenance

Frequency / trigger	Action
Each app startup	Validate required OpenBao configuration and TLS trust; reject insecure schemes, missing paths, invalid credentials, and development fallback in production-like mode.
Each release/update	Pin and review the OpenBao version; review policy changes; run unit/API/security tests and migration consistency checks.
Periodic	Inspect audit logs, token/role lifetime, disk capacity, snapshot freshness, dependency vulnerabilities, and permissions on local data/config files.
After factor revocation	Verify secret deletion/disablement and that the factor is unusable; clear stored references only after cleanup succeeds, with retry state where necessary.
After recovery or laptop migration	Revalidate unseal material, restore process, TLS trust, access policy, token revocation and backup freshness.

11. FIN-14 acceptance and test evidence matrix

The table separates current application evidence from acceptance that remains to be generated. Planned OpenBao tests start as NOT RUN. Do not mark the FIN-14 parent accepted when only the pre-existing MFA endpoints and mocked secret-store tests pass.

Acceptance area	Current evidence in supplied checkpoint	New evidence required for OpenBao	Status
MFA application lifecycle	MFA enrollment/activation/verification /factor listing/revocation code and focused tests reported.	Re-run end-to-end flow using OpenBao adapter; prove pending-to-active gating, replay controls and ownership.	Partial
Secret storage / DB separation	`secret_ref` metadata and provider interface exist; dev encrypted file backend reported.	Inspect database and logs for absence of TOTP seed/URI; write/read/cleanup against KV v2.	Partial
IAM / access policy	No OpenBao policy execution evidence supplied.	Positive test for allowed prefix; negative tests for unrelated path, list, policy/admin APIs and wrong token.	Not run
Encryption / TLS	Development file backend uses AES-GCM per supplied code excerpt; no OpenBao TLS evidence supplied.	Verify TLS certificate validation from Python; test invalid CA rejection; review file permissions and persisted storage design.	Not run
Token life cycle	No AppRole/short-lived token evidence supplied.	Test login, restricted policy, expiry, renewal/re-authentication and revoked secret ID.	Not run
Provider failure / fail-closed	Release notes say production selection rejects unconfigured secret provider.	Stop/seal OpenBao or revoke token; confirm protected login fails safely without fallback.	Not run
Audit / correlation	Application correlation-ID propagation and app auth audit are described as tested; OpenBao audit device not evidenced.	Correlate app event with OpenBao audit operation; inspect redaction and audit-device failure path.	Partial

Acceptance area	Current evidence in supplied checkpoint	New evidence required for OpenBao	Status
Rotation and revocation	Retryable cleanup and cross-user revocation rejection reported in the local checkpoint.	New-factor rotation test, OpenBao data soft-delete, retry and cleanup reference clearing.	Partial
Backup / restore	No OpenBao backup/restore evidence supplied.	Snapshot, restore to isolated instance, unseal, retrieve test secret and verify policy remains intact.	Not run
Overall FIN-14	Latest supplied explicit checkpoint says PARTIAL; full acceptance not verified.	Complete applicable criteria, attach run outputs and reviewer approval.	PARTIAL

11.1 Suggested OpenBao-specific test cases

Test ID	Test	Expected result
TC_OB_001	Persistent secret survives a normal server restart and unseal.	Seed remains retrievable by authorized test principal; no data is lost.
TC_OB_002	FastAPI AppRole reads a secret within the MFA prefix.	Read succeeds and the expected KV v2 payload/version is returned.
TC_OB_003	AppRole tries to read a secret outside the MFA prefix.	Permission denied; no unrelated value is returned.
TC_OB_004	AppRole attempts policy/admin/list/destroy operations.	All operations not required by the service are denied.
TC_OB_005	Enrollment round-trip: create pending factor, scan/enter code, activate.	Only confirmed factor becomes active; DB has reference, no seed.
TC_OB_006	Corrupt/invalid CA or non-TLS endpoint is configured.	Startup/operation fails closed; no insecure retry.
TC_OB_007	OpenBao is sealed/down or token expired during MFA login.	Controlled service error; no MFA bypass and no dev-file fallback.
TC_OB_008	Revoke a factor when OpenBao delete fails, then retry cleanup.	Factor stays revoked; reference is retained until cleanup succeeds, then cleared.
TC_OB_009	User A attempts to revoke/read User B factor through API.	Denied before secret operation; no state or secret changes.
TC_OB_010	Search application and OpenBao logs for a synthetic canary seed.	No plaintext seed, TOTP provisioning URI, AppRole secret ID or token is found.
TC_OB_011	Rotate MFA by enrolling and confirming a second factor.	New factor works; old factor continues only until successful cutover, then is revoked.
TC_OB_012	Snapshot and restore OpenBao + corresponding PostgreSQL test backup.	Restored app can read its test factor and denied-access tests still pass.
TC_OB_013	Disable audit device / simulate disk-full condition in isolated test.	Configured fail/alert policy is followed; no false claim that the failed event was audited.

12. Implementation sequence and definition of done

Phase	Work product	Exit gate
0. Confirm contracts	Review `SecretStore`, MFA service and factor schema in the current branch. Decide version-reference shape and local OpenBao version.	No competing secret path formats; rollback plan for provider selection is agreed.
1. Local service	Persistent config, TLS listener, Raft storage, initialization/unseal runbook, restricted directories.	Data survives restart; listener is loopback-only and TLS-verified.
2. OpenBao bootstrap	KV v2 mount, AppRole, least-privilege policy, audit device, canary secret and denied-access checks.	AppRole cannot access unrelated paths or admin endpoints.
3. Python adapter	OpenBao provider implementing `put/get/delete`, error translation and safe token lifecycle.	Provider contract tests pass; tokens and payloads are not logged.
4. MFA service wiring	Select OpenBao adapter for the secured local profile; use fake provider in unit tests.	Enrollment, activation, verification, revocation and cleanup tests pass.
5. Failure and security tests	Outage injection, cross-user ownership, DB compensation, TLS-negative, token expiry and audit checks.	All critical tests pass; no critical security defects.
6. Recovery proof	Raft snapshot and PostgreSQL backup; isolated restore and access-policy validation.	Restore report includes steps, measured duration, recovery caveats, RTO/RPO decision.
7. Acceptance package	Evidence bundle, exact FIN-14 criteria map, screenshots/log excerpts redacted of secrets, version/policy inventory.	Reviewer signs off; FIN-14 changed from PARTIAL only when every applicable criterion has evidence.

Definition of Done for the OpenBao integration

- OpenBao is running in persistent mode and configured outside the repository; its version and local startup procedure are documented.
- The application authenticates using a restricted machine identity; root/unseal material is never used by FastAPI and never appears in Git.
- Secrets are stored in KV v2; Postgres contains only references/metadata; no seed or provider token appears in logs or API responses.
- FastAPI handles OpenBao failure with bounded timeouts and safe errors; MFA-protected authentication never bypasses MFA.
- Application and OpenBao audit evidence correlate and are protected according to the documented local threat model.
- Positive, negative, lifecycle, rotation, outage and backup/restore tests pass; actual test output and version details are attached.
- The implementation preserves existing login throttling, session, audit immutability, correlation ID and cross-user authorization behavior.
- FIN-14 evidence is reviewed and accepted; until then, status remains PARTIAL.

13. Risks, trade-offs, and residual limitations

Risk	Why it matters	Mitigation / accepted limitation
Single Mac is a single point of failure	If the Mac or OpenBao is down, MFA seed retrieval may fail and protected login cannot continue.	Accept for local-only target; fail closed; define recovery target and rehearse restore. No claim of HA.
Unseal material lost	Encrypted persistent data may be unrecoverable.	Separate protected recovery materials; verify operator runbook and restore drill.
FastAPI token has access to all MFA paths	A compromised API process can read multiple users' seeds within its policy scope.	Limit to MFA prefix; validate owner/factor in application; no browser access; short token TTL; audit access. Stronger isolation would require a separate trust boundary.
Static AppRole secret ID exposure	An attacker with the ID plus Secret ID may authenticate within the role policy.	Store Secret ID in Keychain or protected file; short TTL / usage constraints where feasible; rotate and revoke; restrict local OS access.
Audit log altered by local administrator	File logs on the same machine are not immutable against a privileged operator.	Document threat model; restrictive permissions, checksums and encrypted offline copy; if immutability is a hard requirement, use a separate protected destination or revise the local-only constraint.
Cross-store partial failure	OpenBao and PostgreSQL cannot share one ACID transaction.	Use idempotent operations, compensation, retained cleanup references and a reconciliation test.
Provider-specific API mismatch	A Vault-compatible Python client may not automatically cover every OpenBao behavior/version.	Use the documented OpenBao HTTP API via a standard HTTP client or test the chosen client against the pinned release.
Secret path access is broad at service layer	Prefix policy limits namespace but not individual end-user paths when the same service token serves all users.	Use opaque server-only paths and robust ownership checks; do not claim per-user OpenBao ACL isolation.

13.1 Trade-off assessment

OpenBao is more operationally demanding than keeping an encrypted file, but it gives the project a dedicated API boundary, a centralized policy model, version lifecycle, machine authentication, and audit events. For a local project that wants a controllable, open-source and reusable secret-management component, that trade-off is justified if the operator runbook, backup, and restore tests are implemented. If the project will never need multiple secrets or policy/audit evidence, the extra service could be considered overengineering; however, FIN-14 specifically requires production-provider readiness, so a real secret store is a reasonable architectural choice.

14. Conclusion

OpenBao is the recommended permanent local secrets provider for FinSight AI. The design should place it beside FastAPI and PostgreSQL on the development Mac: PostgreSQL remains authoritative for users, sessions, MFA-factor metadata and secret references; OpenBao stores TOTP seeds and other static secret values; FastAPI is the only application component allowed to call OpenBao; and the frontend never receives OpenBao credentials.

Existing evidence shows MFA routes, factor metadata, a provider-neutral interface, development/test adapters, and substantial local test coverage. However, OpenBao itself is not evidenced as installed or integrated; production currently fails closed without a provider. Implement persistent OpenBao, TLS, least-privilege policy, AppRole, auditing, failure handling, and backup/restore tests before accepting FIN-14. Use a pinned OpenBao version, keep root and unseal material separate, and retain FIN-14 = PARTIAL until end-to-end and recovery evidence is attached.

Appendix A. Illustrative policy and configuration variables

The snippets below are examples of the intended configuration shape. Validate exact syntax against the OpenBao version pinned for this project before applying. Never paste live tokens, role secret IDs, unseal shares or real TOTP seeds into this report, Git, a terminal transcript, or an issue tracker.

A.1 Example policy boundary

```
# finsight-mfa.hcl (illustrative; KV v2 mount named secret/)
path "secret/data/finsight/mfa/*" {
  capabilities = ["create", "read", "update", "delete"]
}

# Deliberately omit access to:
# - sys/* administration endpoints
# - auth/* policy and auth-method configuration
# - unrelated secret namespaces
# - broad list / metadata-destroy capabilities
```

The service policy needs only the capabilities required by implemented lifecycle operations. If deletion is moved to an operator/reconciliation identity, remove `delete` from the runtime policy. Test that the AppRole cannot list or read unrelated values. The path policy is service-level least privilege; it does not replace FinSight's factor-owner checks.

A.2 Example application configuration placeholders

```
# .env.example only - placeholders, no credentials
APP_ENV=local-secure
SECRET_STORE_PROVIDER=openbao
OPENBAO_ADDR=https://127.0.0.1:8200
OPENBAO_CA_CERT=/absolute/path/to/local-openbao-ca.pem
OPENBAO_KV_MOUNT=secret
OPENBAO_MFA_PREFIX=finsight/mfa
OPENBAO_APPROLE_ROLE_ID=<role-id-placeholder>
OPENBAO_APPROLE_SECRET_ID_SOURCE=macos-keychain
OPENBAO_CONNECT_TIMEOUT_SECONDS=2
OPENBAO_READ_TIMEOUT_SECONDS=3
```

The Role ID is not equivalent to a password, but should still be treated as configuration. The Secret ID is sensitive and must come from the protected local credential source. Avoid storing `OPENBAO_TOKEN` as a long-lived root token in `.env`. TLS verification must remain enabled.

A.3 Example adapter error contract

```
OpenBao adapter raises typed internal errors, for example:
  SecretStoreConfigurationError  -> invalid/missing config or TLS trust
  SecretStorePermissionError     -> 403 / policy denied
  SecretNotFoundError            -> 404 / missing version
  SecretStoreUnavailableError    -> timeout, connection failure, sealed service

FastAPI response mapping (illustrative):
  invalid client input or wrong owner -> 4xx without secret details
  secret provider unavailable         -> controlled 503 + correlation ID
  MFA verification failure            -> generic authentication failure

Logs contain event, operation, path identifier (appropriately minimized),
provider status class and correlation ID - never payload or token values.
```

Appendix B. References and source traceability

B.1 Supplied FinSight AI project sources

[S1] FinSight Software Requirements Specification (SRS), supplied PDF.

Relevant: system architecture (pp. 1-2); authentication and access requirements (pp. 3-5), including FR-P1-09 through FR-P1-12 and NFR-P1-01/NFR-P1-02.

[S2] FinSight AI Technical Design & Implementation Plan, supplied PDF.

Relevant: modular FastAPI architecture and repository boundaries (pp. 5-9); security threat controls and secrets manager/database-reference rule (p. 14); reliability, backups, recovery and RTO/RPO ownership (pp. 15, 24).

[S3] FinSight AI Database Schema and ERD Reference, supplied PDF.

Relevant: PostgreSQL as structured source of truth, relational ownership/permission/provenance and implementation boundary (pp. 1-2). The current MFA factor schema details cited here are from the supplied implementation evidence.

[S4] Supplied project implementation and test-status excerpts, current conversation attachment.

Checkpoint dated 23 October 2026: MFA routes/repositories; `mfa_factors.secret_ref`; fake and encrypted development file providers; production provider absent and production selection fails closed; focused MFA tests (30 passed) and full API suite (150 passed, 0 failed, 25 warnings); FIN-14 still PARTIAL.

[S5] FinSight AI Sprint 1 Test Plan and Execution Register, supplied PDF.

Relevant: authentication/security testing, environment and secret handling, test-evidence discipline, and requirement that actual result/status only be marked after execution (pp. 1-3 and relevant security cases).

B.2 Official OpenBao documentation

[5] OpenBao Documentation - overview and concepts — https://openbao.org/docs/

Project-wide feature and configuration reference; docs currently show the 2.7.x line.

[6] OpenBao - KV secrets engine v2 — https://openbao.org/docs/secrets/kv/kv-v2/

Versioned key-value secret engine and policy semantics.

[7] OpenBao - Authentication concepts — https://openbao.org/docs/concepts/auth/

Client authentication, policy-associated tokens and token leases.

[8] OpenBao - AppRole authentication method — https://openbao.org/docs/next/auth/approle/

Machine-oriented authentication model and role constraints; validate exact endpoint/settings against the pinned release.

[9] OpenBao - Audit devices — https://openbao.org/docs/audit/
Request/response auditing, errors and multiple-device considerations.

[10] OpenBao - Seal/Unseal — https://openbao.org/docs/next/concepts/seal/
Sealed-state behavior, unseal-key protection and recovery principles.

[11] OpenBao - Integrated Storage (Raft) and storage/backup concepts — https://openbao.org/docs/configuration/storage/raft/
Persistent integrated storage, snapshot and restore guidance; a single node is not a high-availability cluster.

[12] OpenBao - Security model — https://openbao.org/docs/internals/security/
Client/server/storage trust boundaries and TLS-authenticated API transport.

All OpenBao-specific recommendations in this report are based on the linked official documentation plus the supplied FinSight AI sources. Configuration examples are illustrative until validated against the exact OpenBao release installed locally.